

RS for ADD Unit

Source 1			Source 2		
V	Tag	Value	V	Tag	Value
a					
b					
c					
d					

Register	Valid	Tag	Value
R1	1		1
R2	1		2
R3	1		3
R4	1		4
R5	1		5
R6	1		6
R7	1		7
R8	1		8
R9	1		9
R10	1		10
R11	1		11

Frontend Register File

RS for MUL Unit

Source 1			Source 2		
V	Tag	Value	V	Tag	Value
x					
y					
z					
t					

Register	Value
R1	1
R2	2
R3	3
R4	4
R5	5
R6	6
R7	7
R8	8
R9	9
R10	10
R11	11

Architectural Register File

RS for ADD Unit

Source 1			Source 2		
V	Tag	Value	V	Tag	Value
a					
b					
c					
d					

Register	Valid	Tag	Value
R1	1		1
R2	1		2
R3	1		3
R4	1		4
R5	1		5
R6	1		6
R7	1		7
R8	1		8
R9	1		9
R10	1		10
R11	1		11

Frontend Register File

RS for MUL Unit

Source 1			Source 2		
V	Tag	Value	V	Tag	Value
x					
y					
z					
t					

Register	Value
R1	1
R2	2
R3	3
R4	4
R5	5
R6	6
R7	7
R8	8
R9	9
R10	10
R11	11

Architectural Register File

Reorder Buffer (ROB)

Entry 0			
Entry 1			
Entry 2			
Entry 8			
Entry 13			
Entry 14			
Entry 15			

Reorder Buffer (ROB)

Entry 0			
Entry 1			
Entry 2			
Entry 8			
Entry 13			
Entry 14			
Entry 15			

→ this bottom design is used most for modern OoO

at decode: allocate Dest-PR control signal to dest. register
Read and update Frontend reg. map

before EXE: access physical RF to get sources
after EXE: access physical RF to write results

at retirement: update arch. reg. map with Dest-PR

& why is OoO beneficial?

→ Latency Tolerance → tolerates latency of multi-cycle ops. by executing independent ops. concurrently

→ scalability of this is limited by instruction window size (How many running ins.? eventually we start stalling)

Advantages: latency tolerance → higher performance than in-order, irregular parallelism → dynamically finds/exploits par. operations in a program

Disadvantages: higher-complexity → can lengthen critical path delay and therefore TC, more HW resources needed

Exploiting irregular parallelism with dataflow: → this is its major strength, but it has downsides that make it unviable at ISA level for now:

→ no precise state semantics → debugging much harder, same for exceptions

→ large HW overhead (tag matching, data/tag storage)

→ no control over parallelism, another challenge is using mutable data structures

→ Dataflow is still a success at the mach level → OoO is dataflow, Von Neumann model semantics are still complied with

Superscalar Execution:

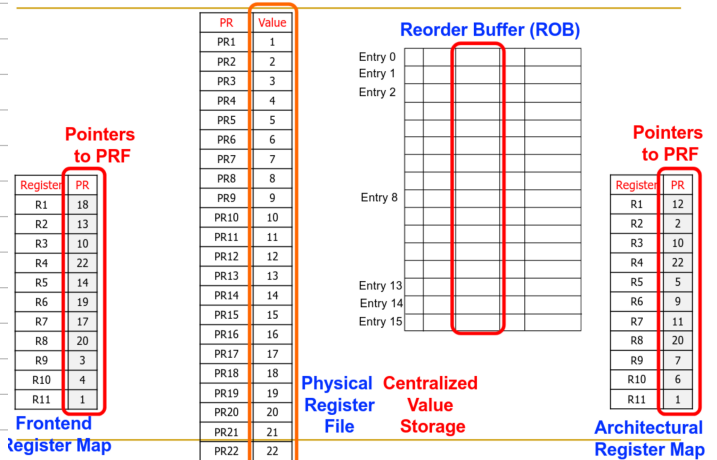
Idea: spatial parallelism & fetch, decode, execute, retire multiple ins. per cycle → N-wide superscalar = N ins. per cycle

→ Hardware checks concurrently fetched ins. for dependencies

→ orthogonal to OoO, can go hand-in-hand

→ In-Order Superscalar: multiple copies of datapath

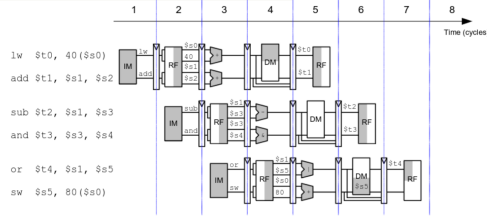
Getting Rid of Replicated Values



→ Performance Example:

```
lw $t0, 40($s0)
add $t1, $s1, $s2
sub $t2, $s1, $s3
and $t3, $s3, $s4
or $t4, $s1, $s5
sw $s5, 80($s0)
```

Ideal IPC = 2



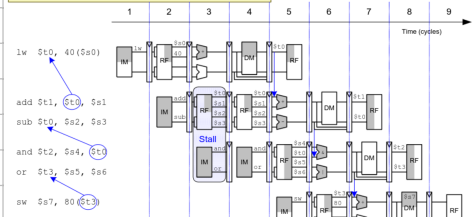
Actual IPC = 2 (6 instructions issued in 3 cycles)

Superscalar Performance with Dependencies

```
lw $t0, 40($s0)
add $t1, $t0, $s1
sub $t0, $s2, $s3
and $t2, $s4, $t0
or $t3, $s5, $s6
sw $s7, 80($t3)
```

Ideal IPC = 2

Can you reorder the instructions to get IPC = 2?



Actual IPC = 1.2 (6 instructions issued in 5 cycles)

→ we can still apply all of the fundamental ways to handle dependencies: stall, forward, eliminate in software, reorder, predict and verify / or used fine-grained multithreading

Advantage of Superscalar: higher ins. throughput → CPI becomes IPC and the IPC can be much higher

Disadvantages: Complexity for dependence checking → check between concurrent ins., register renaming becomes more complex / critical path delay can be longer → T_c higher

→ More HW resources needed

Handling Control Dependencies:

→ moreover, determining the next PC, the problems are: determining it for control flow ins. and determining if we have control flow ins.

Branch Types

Type	Direction at fetch time	Number of possible next fetch addresses?	When is next fetch address resolved?
Conditional	Unknown	2	Execution (register dependent)
Unconditional	Always taken	1	Decode (PC + offset)
Call	Always taken	1	Decode (PC + offset)
Return	Always taken	Many	Execution (register dependent)
Indirect	Always taken	Many	Execution (register dependent)

→ we could stall for these dependencies, but once again it's inefficient

→ The Branch Problem:

→ 15-25% of all ins. are control flow

→ Problem: next PC after control flow ins. is not determined after N cycles in a pipelined processor

→ fetching W ins. per cycle, a misprediction wastes $N \times W$ ins. slots

Branch Prediction:

Simplest method: always guess $\text{NextPC} = \text{PC} + 4$

→ how can we make this more effective?

→ Idea: maximise chances that next ins. to be executed is the next seq. ins.

→ in SW: lay out control flow graph s.t. "likely next ins." is an not taken

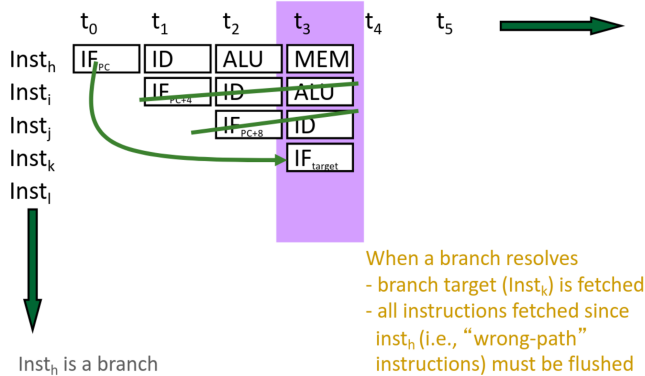
side of the branch & profile guided code positioning, in HW: cache traces of executed ins. (later)

→ Another way to make our guess of $\text{PC} + 4$ more effective: minimize control flow ins.

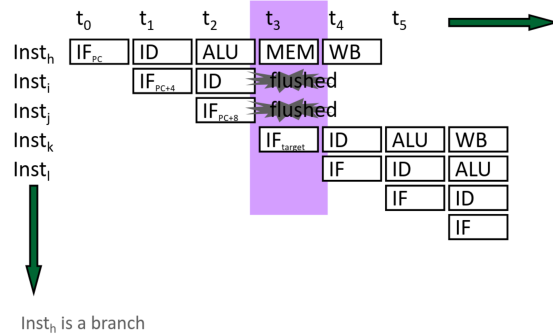
→ How? 1. remove unnecessary control flow ins. → combine predicates, test once, 2. turn control dep. to data dep. → predicated execution (if conversion)

→ How? 1. remove unnecessary control flow ins. → combine predicates, test once, 2. turn control dep. to data dep. → predicated execution

how branching looks: branch taken:



misprediction:



Performance Analysis: correct guess $\rightarrow$ no penalty, incorrect guess $\rightarrow$ 2 bubbles

Assume: no data dependence stalls, 20% of ins. are control flow, 20% of these are taken

$$CPI = [1 + (0,2 \cdot 0,2) \cdot 2] = [1 + 0,08] = 1,08 \rightarrow \text{how do we reduce these?}$$

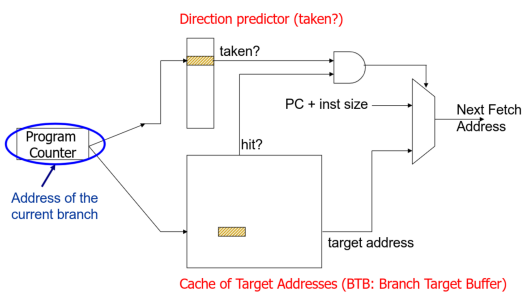
(wrong guess) $\rightarrow$ penalty

Idea: predict next fetch address $\rightarrow$ used in next cycle, no wasted cycles on prediction

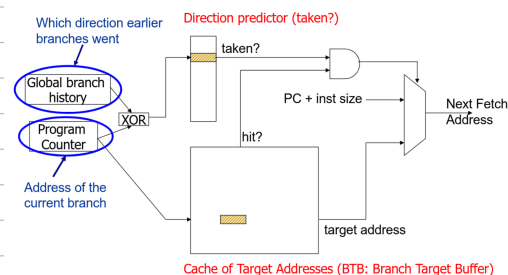
$\rightarrow$ 3 things need to be predicted: is the fetched ins. a branch, branch direction, branch target address

$\rightarrow$ Observation: Target address remains same for conditional direct branch across instances

$\rightarrow$ we store the target of previous branches in the Branch Target Buffer (BTB)



more advanced:



$\rightarrow$ 1 and 3 from before are solved with a BTB, but how do we predict if a branch is taken?

Static Branch Prediction (at compile time)

$\rightarrow$ always not taken $\rightarrow$ simple to implement (without BTB), 50-40% acc., compiler can still lay out code so less branches are taken (helpful)

$\rightarrow$ always taken $\rightarrow$ 60-70% acc. (for conditional)

$\rightarrow$ backward branch: target addr. lower than branch PC $\rightarrow$ usually taken (example: loop branches)

$\rightarrow$ BTBN (backwards taken, forwards not taken)

$\rightarrow$ profile based: compiler determines likely direction for each branch, encodes it as hint bit in branch ins. format

$\rightarrow$ program based: heuristics based on program analysis provide statically predicted direction $\rightarrow$ 20% mispred. rate

$\rightarrow$ example: mark certain opcodes as not taken and loops as taken

$\rightarrow$ programmer-based: pragmas (keywords) in programming language allow programmer to categorize branches

one can say these push the problem up to SW, requiring compiler and ISA support

→ common disadvantage of static prediction: cannot adapt to dynamic changes in branch behavior

Dynamic Branch Prediction: collect information about branches at runtime

Advantages: prediction based on history of branch execution → adapt better, no need for static profiling and support from compiler for this...

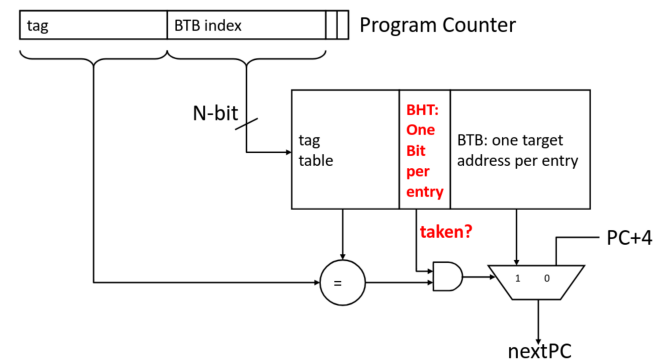
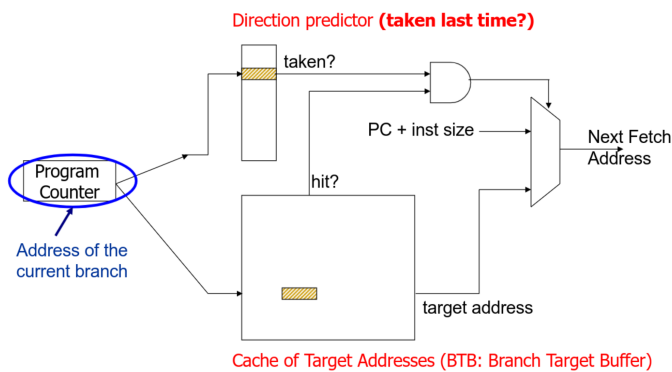
Disadvantage: adds HW complexity

Last time predictor: guess that a branch has same behavior as last instance → single bit stored in BTB per branch

→ always misses first and last iteration of loop branch → accuracy for N iterations → $(N-2)/N$

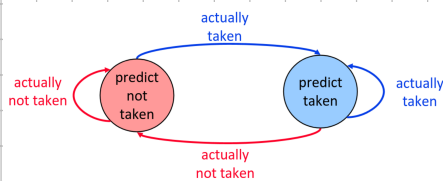
→ high accuracy for large N loops, much lower for small N

Implementation:



The 1-bit BHT (Branch History Table) entry is updated with the correct outcome after each execution of a branch

→ FSM:

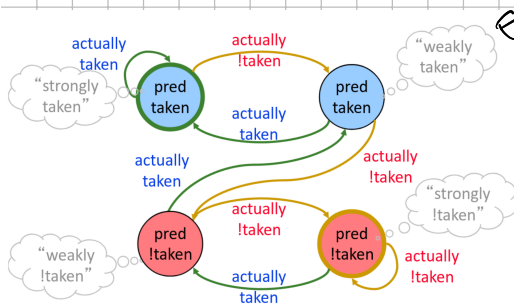


→ problem: too fickle, solution: add hysteresis, don't let single outcome

Flip our expectation → 2 bits per branch in BTB instead of one (bimodal)

→ better accuracy, but higher HW cost
→ ~85-90%

→ we can do better!



← bimodal FSM

→ global branch correlation: recently executed branch outcomes in the exec. path we correlated with the outcome of the next branch

example:

→ Idea: associate branch outcomes with global "taken/not taken history"

→ predict based on outcome of branch last time we had same global branch history

→ implementation: keep track of global history in register (GHR) (first level)

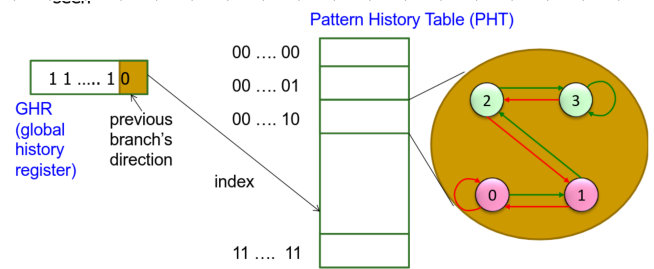
→ use GHR to index into table containing outcomes for each global history, stored bimodally (second level)

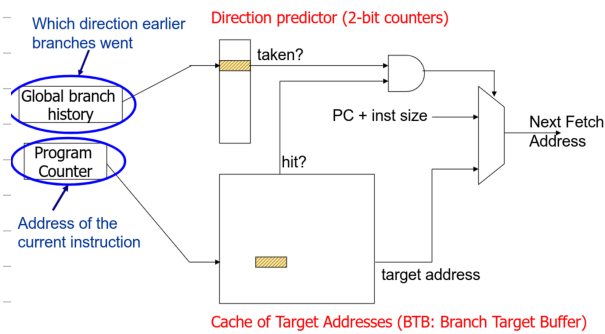
→ two-level global branch prediction

Example:

for (i=0; i<100; i++)
for (j=0; j<3; j++)

test	value	GR	result
j<3	j=1	1101	taken
j<3	j=2	1011	taken
i<100	i=3	0111	not taken
i<100		1110	usually taken

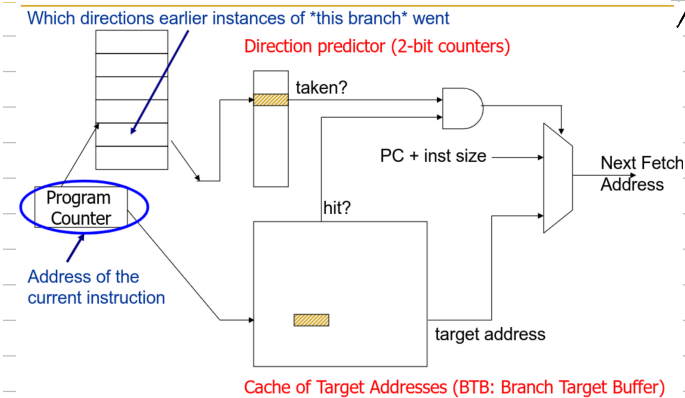
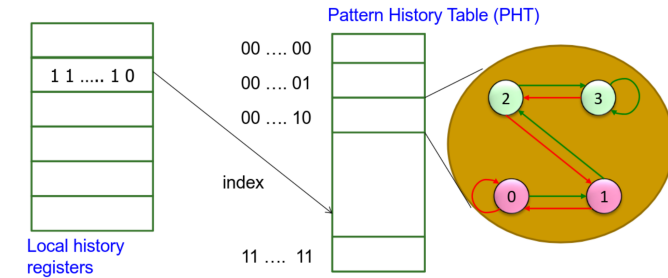
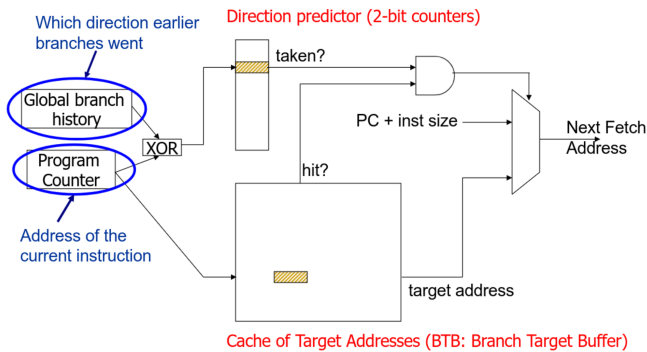




→ improving its accuracy:

- Idea: add more context information to global predictor depending on ^(specific) branch
- Gshare: flush the GHR with the branch PC
- more context, better usage of 2-bit counter array, but increases latency
- Local branch correlations: a branch's outcome can be correlated to its own past outcomes

Two-Level Gshare Branch Predictor



- Perfect loop prediction: separate PHT entry for each local history lets us distinguish loops by their iteration → works for short loops
- Idea: per-branch history → we implement the local history predictor with 2 levels like GHR
- level 1: local history registers (N bits each) → select history reg. based on PC of branch
- level 2: table of counters for each entry → direction of this branch with this exact history

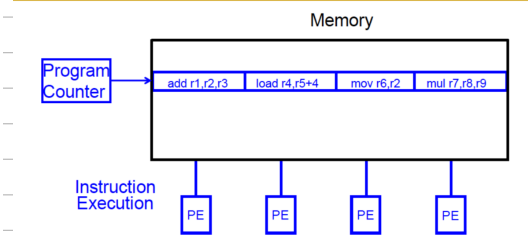
- can we do better? yes, the predictability of branches varies and there is no one-size-fits-all prediction algorithm
- Idea: exploit this using hybrid branch predictors
- Advantages:
 - better acc. → different predictors for different branches
 - reduced warmup time (we can use faster-warmup predictor in beginning)

- Disadvantages: we need a meta-predictor to predict the predictor, longer access latency, more HW and complexity
- Biased Branches: some branches pollute prediction structures because they are strongly biased in one direction
- solution: detect biased branches, use simpler predictor for them
- Branch prediction is not a solved problem (state-of-the-art: 97%) → we have more advanced techniques like loop detectors and perceptron predictors that I will skip over because I doubt they will be on the exam

Very Large Instruction Word (VLIW)

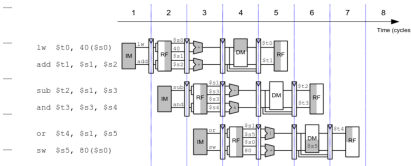
- Idea: SW packs ind. ins. into an ins. bundle that is fetched and executed concurrently
- no need to check HW dependencies between concurrently-fetched ins.

VLIW Concept



lw \$t0, 40(\$s0)	add \$t1, \$s1, \$s2	Bundle 1
sub \$t2, \$s1, \$s3	and \$t3, \$s3, \$s4	Bundle 2
or \$t4, \$s1, \$s5	sw \$s5, 80(\$s0)	Bundle 3

Ideal IPC = 2



Actual IPC = 2 (6 instructions issued in 3 cycles)

→ compiler bundles independent ins.

→ VLIW characteristics:

1. multiple ins. fetch/execute, multiple functional units
2. all ins. in a bundle are executed in lockstep (one ins. stalls whole bundle)
3. ins. in a bundle are statically aligned to be directly supplied into FU

→ in free VLIW: compiler handles all dependence-related stalls

Advantages:

→ no need for dynamic scheduling, HW → simple HW

→ no need for dependence checking within VLIW ins. → simple HW, no renaming

→ no need for ins. distribution to different FU → simple HW

Disadvantages:

→ Compiler has to find N ind. ops. per cycle → if it cannot, NOP's are inserted → larger code, less parallelism

→ recompilation required when exec. width, ins. latencies, or FU's change

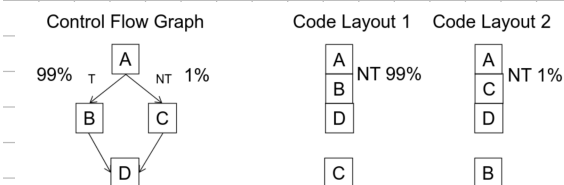
→ lockstep exec. causes ind. ins. to stall → hard to bundle variable latency ops. like memory ops.

→ Observations: static schedule intimately tied to arch, VLIW compiler optimizations generally made compilers faster for superscalar, VLIW is very successful when compiler can find parallelism (like GPUs)

Basic block reordering:

→ Likely-taken branches make static code reordering difficult & convert to likely not-taken branch

→ reorder basic blocks → code with single entry and single exit point

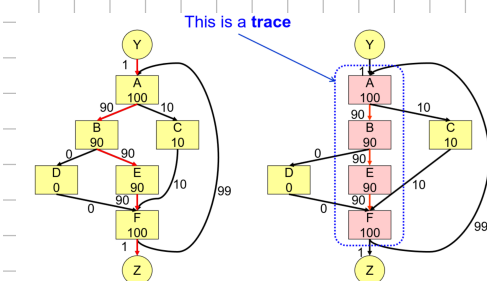


Code Layout 1 leads to the fewest branch mispredictions

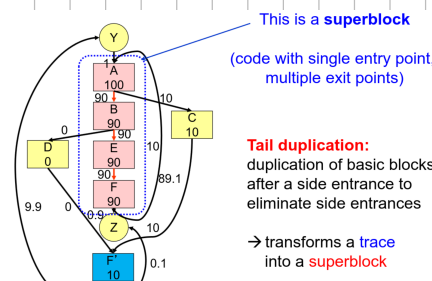
→ Idea: Superblock → combine often taken basic blocks s.t. they form single-entry, multiple-exit larger block → executed as serial code

→ reduces misprediction and enables aggressive compiler optimization, (adv.)

but increases code size, requires recompilation, depends on profile (disadv.)



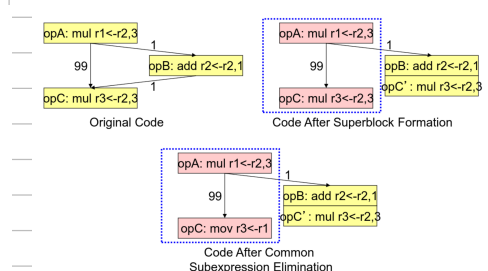
This is a trace



This is a superblock
(code with single entry point,
multiple exit points)

Tail duplication:
duplication of basic blocks
after a side entrance to
eliminate side entrances

→ transforms a trace
into a superblock



Systolic Arrays;

- execution model different from von Neumann and dataflow
- designed as special-purpose accelerators, heavily used in ML / image / signal processing, ... , but can be generalized
- goal: design accelerator with simple, regular design / high concurrency, and balanced computation and I/O bandwidth
- replace single processing element (PE) with array of PEs, orchestrate flow of data between those
- goal: collectively operate on data before outputting to memory

Systolic Arrays

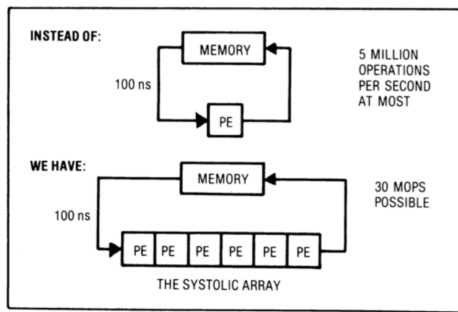


Figure 1. Basic principle of a systolic system.

→ difference from pipelining:

- all PEs are individual, array structure can be non-linear and multi-dimensional
- PE connections can be multidirectional (and different speed)
- PEs can have local memories and execute kernels

Example: Convolution → crucial in ML, simple formula

Given the sequence of weights $\{w_1, w_2, \dots, w_k\}$ and the input sequence $\{x_1, x_2, \dots, x_n\}$, compute the result sequence $\{y_1, y_2, \dots, y_{n+1-k}\}$ defined by

$$y_i = w_1 x_i + w_2 x_{i+1} + \dots + w_k x_{i+k-1}$$

→ carefully orchestrate when data elements are input and when output is buffered

→ Example of 2D systolic computation:

- Multiply two 3x3 matrices (inputs)
 - Keep the final result in PE accumulators

$$\begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$$

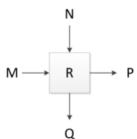


Figure 1: A systolic array processing element

$$\begin{aligned} P &= M \\ Q &= N \\ R &= R + M * N \end{aligned}$$

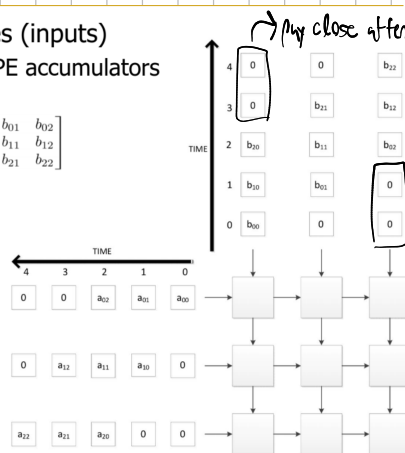


Figure 8. Design W1: systolic convolution array (a) and cell (b) where w_i 's stay and x_i 's and y_i 's move systolically in opposite directions.

→ pay close attention to how data inputs are timed

→ systolic arrays are very powerful when combined & give in-place least squares approximations

Advantages:

- Principled: efficient for limited mem. bandwidth / balances computation to I/O bandwidth
- specialized → in special cases, improve efficiency, simple design / high concurrency / performance

Disadvantage: Specialized → not generally usable, computation needs to fit PE functions / organization

→ Every PE in array can store multiple weights, select in-place

→ take it further → every PE can have own data and ins. memory → you can build pipeline parallelism and staged execution

Pipeline-Parallel (Pipelined) Programs

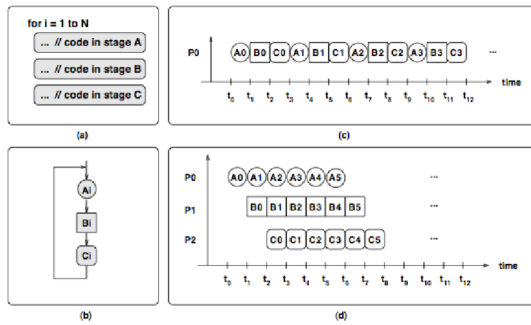
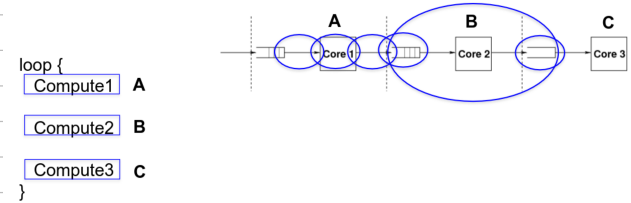


Figure 1. (a) The code of a loop. (b) Each iteration is split into 3 pipeline stages: A, B, and C. Iteration i comprises A_i , B_i , C_i . (c) Sequential execution of 4 iterations. (d) Parallel execution of 6 iterations using pipeline parallelism on a three-core machine. Each stage executes on one core.

→ Loop iterations are divided into code segments called stages

→ threads execute stages on different cores

→ cores may be specialized



SIMD:

Flynn's Taxonomy of Computers; SISO → single ins., single data element (el.), SIMD → single ins., multiple data el.

(found in array and vector processors), MISD → multiple ins., single data el. (systolic arrays closest), MIMD;

multiple ins., multiple data el. (multiprocessor, multithreading)

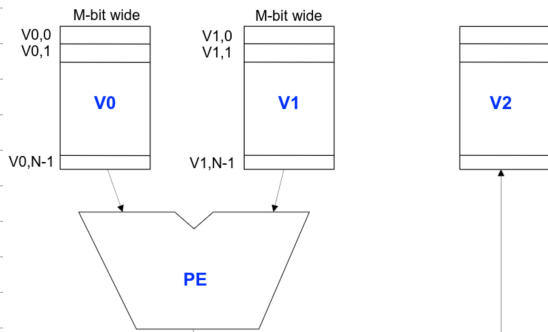
Data parallelism: same operation, different data (SIMD) → dot product (example)

→ what SIMD exploits is called operation-level parallelism → ILP, ins. is same across data

→ we have multiple PEs operating concurrently → time-space duality

→ Array Processor: Ins. operates on multiple el. at once using different PE's (space)

→ Vector Processor: Ins. operates on multiple el. in consecutive time steps using same space (PE)



→ each vector data reg. holds N M-bit values

→ each reg. stores a vector

Instruction Stream

```
LD  VR ← A[3:0]
ADD VR ← VR, 1
MUL VR ← VR, 2
ST  A[3:0] ← VR
```

ARRAY PROCESSOR



Same op @ same time

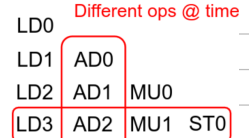


Different ops @ same space

VECTOR PROCESSOR

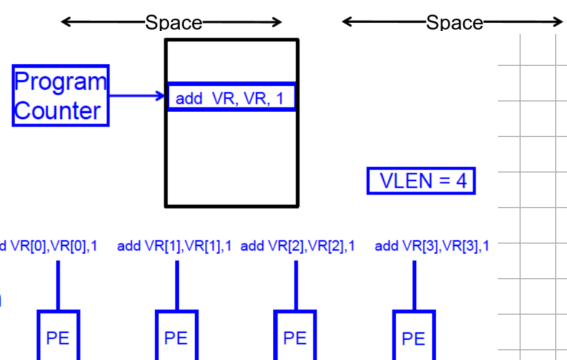
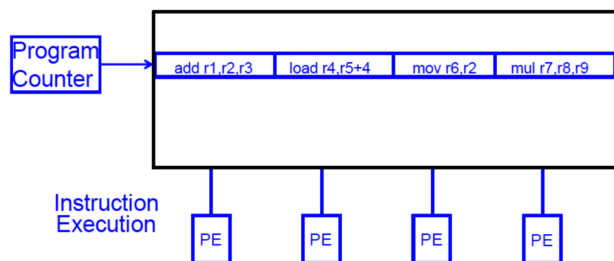


Different ops @ time



Same op @ space

Comparison to VLIW:



Vector Processors:

→ a C++-style vector is a 1D array of numbers → vector processor operates on vectors, not scalars

→ requirements: load/store vectors → vector registers / operate on diff. length vectors → Vector Length Register (VLEN), elements of vector may have distance in mem. → Vector Stride Register (VSTR) → stride refers to this distance

Example:

$$A_{4 \times 6} \quad B_{6 \times 10} \rightarrow C_{4 \times 10}$$

0	1	2	3	4	5
6	7	8	9	10	11

$A_{4 \times 6} \quad B_{6 \times 10} \rightarrow C_{4 \times 10}$

Dot product of each row vector of A with each column vector of B

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20									
30									
40									
50									

→ Load A's row 0 (A0-A5) into vector reg. V1

→ increment address by 1 for next column → stride of 1

→ Load B's column 0 (B0-B50) into V2

→ increment address by 10 for next row → stride of 10

Linear Memory

A	0
	1
	2
	3
	4
	5
	6
B	0
	1
	2
	3
	4
	5
	6
	7
	8
	9
	10

→ vector ins. performs op. on each el. in consecutive cycles → FU's are pipelined

→ vector ins. allow for deeper pipelines: no intra-vector dependencies → no HW interlocking needed
no control flow within vector
known stride → easy address calculation → easy loading (even prefetching) into mem.

Advantages:

→ No dependencies within vector → deep pipelines

→ each ins. generates many operations → reduces fetch bandwidth requirements, amortizes ins. fetch and control overhead over large dataset

→ no need to explicitly code loops → less branches

→ regular mem. access pattern.

→ Disadvantage: only works for regularly parallelizable operations like vector operations, inefficient for irregular parallelism

→ Amdahl's Law: given a parallelizable fraction F of a program and N processors:

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

→ all parallel machines suffer from serial bottleneck

→ memory (bandwidth) can become bottleneck for vector proc.
if → compute / mem. op. balance not maintained
→ data not mapped appropriately to memory banks

→ VMASK register: indicates which elements of vector to operate on → all vector data regs. have NM-bit values

Loading/Storing Vectors from/to Memory:

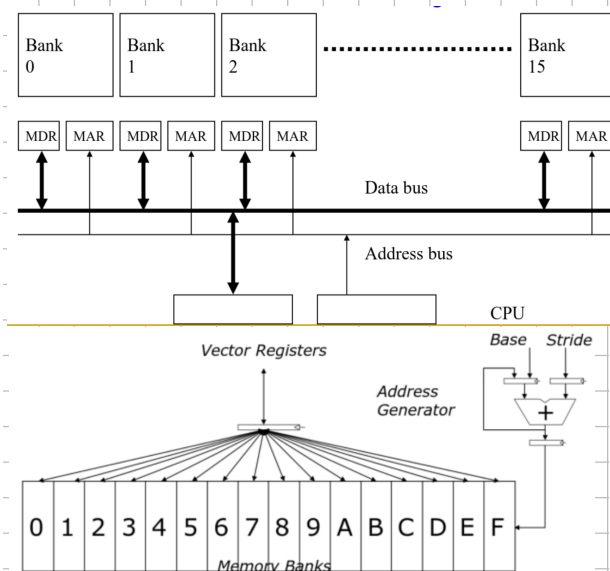
→ load/store multiple el. separated by a constant stride

→ El. can be loaded in consecutive cycles if we can start load of 1 element per cycle → sustain throughput of 1 el. per cycle

→ but memory takes more than 1 cycle to address, what do we do? Bank the memory, interleave elements across banks

→ memory is divided into banks that are accessible independently, banks share address and data buses

→ can start and complete 1 bank access per cycle → sustain N concurrent accesses if all N go to different banks



Vector Memory System:

→ next address = prev. address + stride

→ if (stride == 1) && (consecutive elements interleaved across banks)
&& (number of banks >= bank latency) / then:

→ we can sustain 1 element/cycle throughput

Scalar Code Example: Element-Wise Avg.

■ For I = 0 to 49

□ $C[i] = (A[i] + B[i]) / 2$

■ Scalar code (instruction and its latency in clock cycles)

MOVI R0 = 50	1	
MOVA R1 = A	1	304 dynamic instructions
MOVA R2 = B	1	
MOVA R3 = C	1	
X: LD R4 = MEM[R1++]	11	;autoincrement addressing
LD R5 = MEM[R2++]	11	
ADD R6 = R4 + R5	4	
SHFR R7 = R6 >> 1	1	
ST MEM[R3++] = R7	11	
DECBNZ R0, X	2	;decrement and branch if NZ

Scalar Code Execution Time (In Order)

■ Scalar execution time on an in-order processor with 1 bank

- First two loads in the loop cannot be pipelined: 2*11 cycles
- $4 + 50*40 = 2004$ cycles

■ Scalar execution time on an in-order processor with 1 bank with 2 memory ports (two different memory accesses can be serviced concurrently) OR 2 banks (where arrays B and C are stored in different banks)

- First two loads in the loop can be pipelined: 1 + 11 cycles
- $4 + 50*30 = 1504$ cycles

■ For I = 0 to 49

□ $C[i] = (A[i] + B[i]) / 2$

■ Vectorized loop (each instruction and its latency):

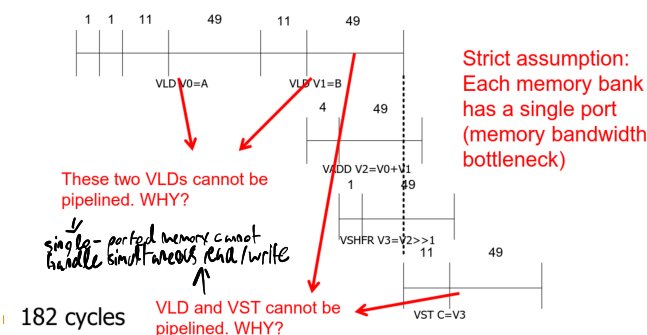
MOVI VLEN = 50	1	7 dynamic instructions
MOVI VSTR = 1	1	
VLD V0 = A	11 + VLEN - 1	
VLD V1 = B	11 + VLEN - 1	
VADD V2 = V0 + V1	4 + VLEN - 1	
VSHFR V3 = V2 >> 1	1 + VLEN - 1	
VST C = V3	11 + VLEN - 1	

→ why 16 banks? 11-cycle mem. access latency

→ $16 > 11 \Rightarrow$ overlap enough mem. ops. to cover mem. latency

→ example assumes stride = 1, what if it's higher?

→ let us use vector chaining → data forwarding from one vector FU to another



182 cycles

→ A loop is vectorizable if each iteration is independent of any other

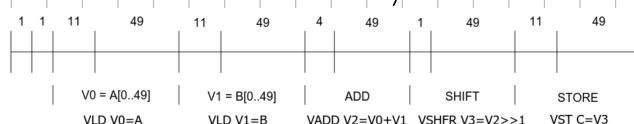
→ Assume no chaining (vector data forwarding)

→ Output of vector FU cannot be used as direct input of another

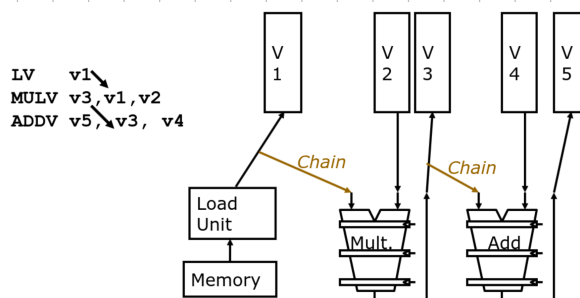
→ entire vector reg. needs to be ready before any of it can be used in other op.

→ assume 1 mem. port per bank and 16 mem. banks

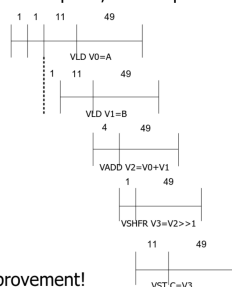
→ word-interleaved (consecutive els. of array stored in consecutive banks)



■ 285 cycles



■ Chaining and 2 load ports, 1 store port in each bank



■ 79 cycles

■ 19X perf. improvement!